

Algorithm Design Strategy

Divide and Conquer

More examples of Divide and Conquer

- Review of Divide & Conquer Concept
- More examples
 - Quicksort
 - Finding closest pair of points
 - Matrix Multiplication Algorithm
 - Large Integer Multiplication
 - 2-D closest pair of points
 - Tromino tiling

Divide and Conquer Approach

- Concept: D&C is a general strategy for algorithm design

It involves three steps:

(1) Divide an instance of a problem into one or more smaller instances

(2) Conquer (solve) each of the smaller instances. Unless a smaller instance is sufficiently small, use recursion to do this

(3) If necessary, combine the solutions to the smaller instances to obtain the solution to the original instances (e.g., Merge sort)

□ Approach:

- Recursion (Top-down approach)

Example 1: Binary Search

□ Binary search in a sorted array with size n

Worst case time complexity in the recursive binary search:

$$W(n) = W(n/2) + 1$$

$$W(1) = 1$$

Solve the recurrence, and obtain:

$$W(n) = \lg n + 1 \in \Theta(\lg n)$$

Example 2: Quicksort

```
quicksort(L)
{
  if (length(L) < 2) return L
  else
  {
    pick some x in L      // x is the pivot element
    L1 = { y in L : y < x }
    L2 = { y in L : y > x }
    L3 = { y in L : y = x }
    quicksort(L1)
    quicksort(L2)
    return concatenation of L1, L3, and L2
  }
}
```

Example 2: Quicksort

Example 2.3

Suppose the array contains these numbers in sequence:

Pivot item



15 22 13 27 12 10 20 25

1. Partition the array so that all items smaller than the pivot item are to the left of it and all items larger are to the right:

Pivot item



10 13 12 15 22 27 20 25
All smaller All larger

2. Sort the subarrays:

Pivot item



10 12 13 15 20 22 25 27
Sorted Sorted

Example 2: Quicksort

Algorithm 2.7

Partition

Problem: Partition the array S for Quicksort.

Inputs: two indices, low and $high$, and the subarray of S indexed from low to $high$.

Outputs: $pivotpoint$, the pivot point for the subarray indexed from low to $high$.

```
void partition (index low, index high,
               index& pivotpoint)
{
    index i, j;
    keytype pivotitem;

    pivotitem = S[low];           // Choose first item for
    j = low;                     // pivotitem.
    for (i = low + 1; i <= high; i++)
        if (S[i] < pivotitem){
            j++;
            exchange S[i] and S[j];
        }
    pivotpoint = j;
    exchange S[low] and S[pivotpoint]; // Put pivotitem at pivotpoint.
}
```

Example 2: Quicksort

• **Table 2.2** An example of procedure *partition**

<i>i</i>	<i>j</i>	<i>S</i> [1]	<i>S</i> [2]	<i>S</i> [3]	<i>S</i> [4]	<i>S</i> [5]	<i>S</i> [6]	<i>S</i> [7]	<i>S</i> [8]	
—	—	15	22	13	27	12	10	20	25	← Initial values
2	1	15	22	13	27	12	10	20	25	
3	2	15	22	13	27	12	10	20	25	
4	2	15	13	22	27	12	10	20	25	
5	3	15	13	22	27	12	10	20	25	
6	4	15	13	12	27	22	10	20	25	
7	4	15	13	12	10	22	27	20	25	
8	4	15	13	12	10	22	27	20	25	
—	4	10	13	12	15	22	27	20	25	← Final values

* Items compared are in boldface. Items just exchanged appear in squares.

Time Complexity of Quicksort

- $T(n) = T(n_1) + T(n_2) + n-1$
 - n_1 = length of L_1 , n_2 = length of L_2
 - $n_1 + n_2 = n - 1$ if all the elements in L are unique
- Best case
 - L is divided into two halves at every recursion
 - $T(n) = 2T(n/2) + n-1 = \Theta(n \lg n)$
- Average case
 - $\Theta(n \lg n)$
 - See the textbook for a proof
- Worst case
 - $T(n) = T(0) + T(n-1) + n-1$, $T(0)=0$ when L is already sorted
(As a result, the pivot is always the smallest)
 - So, $T(n) = \Theta(n^2)$
 - Why is it called quicksort then?

Average-Case Time Complexity of Quicksort

$$\begin{array}{c}
 \text{Probability} \\
 \text{pivotpoint is } p \\
 \downarrow \\
 A(n) = \sum_{p=1}^n \frac{1}{n} \underbrace{[A(p-1) + A(n-p)]}_{\substack{\text{Average time to} \\ \text{sort subarrays when} \\ \text{pivotpoint is } p}} + \underbrace{n-1}_{\substack{\text{Time to} \\ \text{partition}}} \quad (2.1)
 \end{array}$$

In the exercises we show that

$$\sum_{p=1}^n [A(p-1) + A(n-p)] = 2 \sum_{p=1}^n A(p-1).$$

Plugging this equality into Equality 2.1 yields

$$A(n) = \frac{2}{n} \sum_{p=1}^n A(p-1) + n - 1.$$

Multiplying by n we have

$$nA(n) = 2 \sum_{p=1}^n A(p-1) + n(n-1). \quad (2.2)$$

Applying Equality 2.2 to $n - 1$ gives

$$(n - 1) A (n - 1) = 2 \sum_{p=1}^{n-1} A (p - 1) + (n - 1) (n - 2). \quad (2.3)$$

Subtracting Equality 2.3 from Equality 2.2 yields

$$n A (n) - (n - 1) A (n - 1) = 2 A (n - 1) + 2 (n - 1),$$

which simplifies to

$$\frac{A (n)}{n + 1} = \frac{A (n - 1)}{n} + \frac{2 (n - 1)}{n (n + 1)}.$$

If we let

$$a_n = \frac{A (n)}{n + 1},$$

we have the recurrence

$$a_n = a_{n-1} + \frac{2 (n - 1)}{n (n + 1)} \quad \text{for } n > 0$$

$$a_0 = 0.$$

Like the recurrence in Example B.22 in [Appendix B](#), the approximate solution to this recurrence is given by

$$a_n \approx 2 \ln n,$$

which implies that

$$\begin{aligned} A (n) &\approx (n + 1) 2 \ln n = (n + 1) 2 (\ln 2) (\lg n) \\ &\approx 1.38 (n + 1) \lg n \in \Theta (n \lg n). \end{aligned}$$

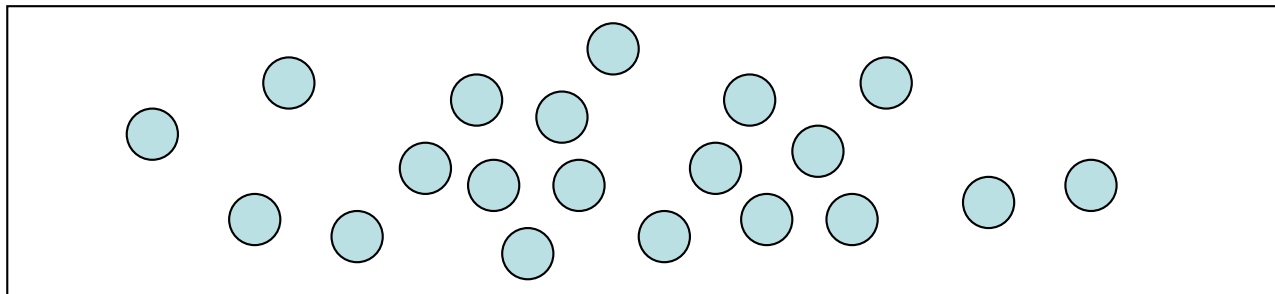
How to make Quicksort efficient

- Randomization
 - Randomly pick an element in L as the pivot
 - So, each item has the same probability of $1/n$ to be selected as the pivot
 - Avoid the worst case (what is the worst case in quicksort?)
 - $O(n \lg n)$ is expected running time
 - proof not required

Example 3: Closest Pair Problem

□ Finding the closest pair of points

- Find the closest pair of points in a set of points. The set consists of points in two dimension plane
- Given a set P of N points, find $p, q \in P$, such that the distance $d(p, q)$ is minimum



- Application:
 - Traffic control systems: A system for controlling air or sea traffic might need to know which two vehicles are too close in order to detect potential collisions.
 - Computational geometry

Brute force algorithm

Input: set S of points

Output: closest pair of points

```
min_distance = infinity
```

```
for each point  $x$  in  $S$ 
```

```
    for each point  $y$  in  $S$ 
```

```
        if  $x \neq y$  and  $\text{distance}(x,y) < \text{min\_distance}$ 
```

```
        {
```

```
            min_distance = dist( $x,y$ )
```

```
            closest_pair = ( $x,y$ )
```

```
        }
```

Time Complexity: $O(n^2)$

1 Dimension Closest Pair Problem

- Brute-force algorithm: Find all the distances $D(p, q)$ and find the minimum distance
 - Time complexity: $O(n^2)$
- 1D problem can be solved in $O(n \lg n)$ via sorting
- But, sorting does not generalize to higher dimensions
- Let's develop a divide & conquer algorithm for 2D problem.

2D Closest Pair Problem

□ Finding the closest pair of points

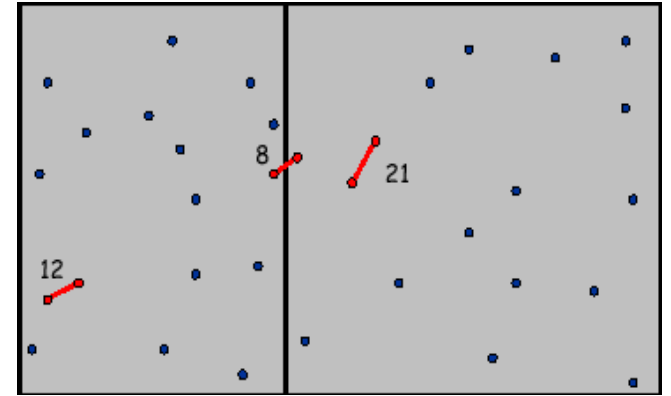
- The "closest pair" refers to the pair of points in the set that has the smallest Euclidean distance,

Distance between points $p_1=(x_1,y_1)$ and $p_2=(x_2,y_2)$

$$D(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

- If there are two identical points in the set, then the closest pair distance in the set will obviously be zero.

2D Closest Pair Problem



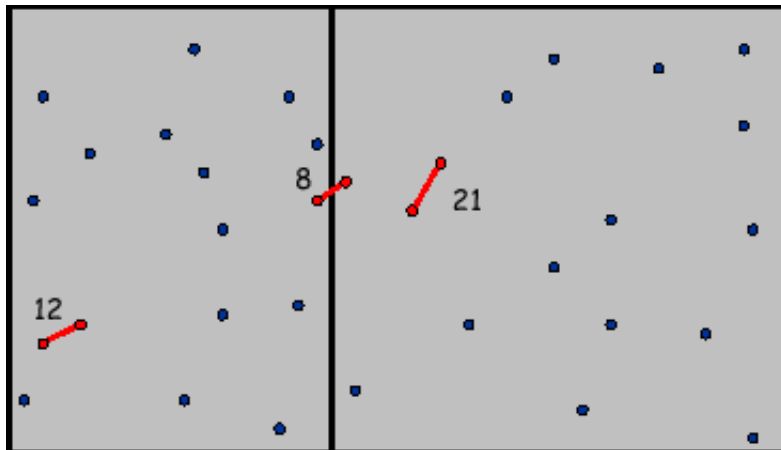
□ Finding the closest pair of points

- Divide: Sort the points by x-coordinate; draw vertical line to have roughly $n/2$ points on each side
- Conquer: Find closest pair in each side recursively
- Combine: Find closest pair with one point in each side
- Return: best of three solutions

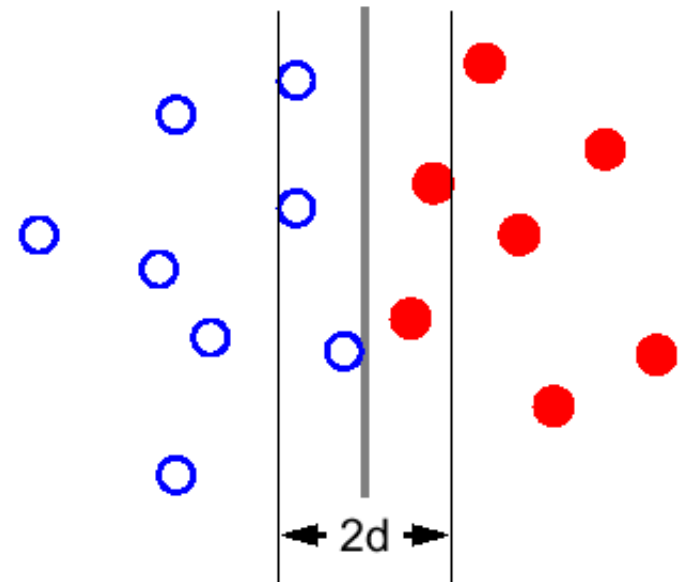
Key observation

Find the closest pair in a strip of width $2d$

Example: $d = \min(12, 21)$

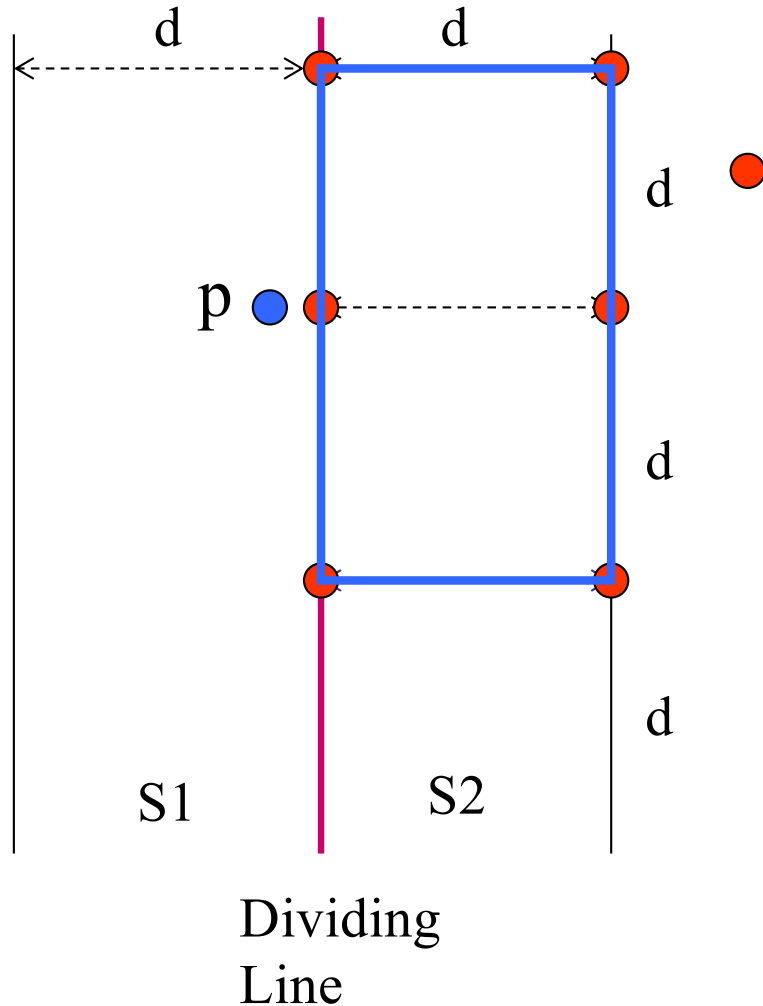


- Find the closest $(\circ, \bullet)$ pair in a strip of width $2d$, knowing that no $(\circ, \circ)$ or $(\bullet, \bullet)$ pair is closer than d .



2D Closest Pair

$$d = \text{minimum}(d_{Lmin}, d_{Rmin})$$



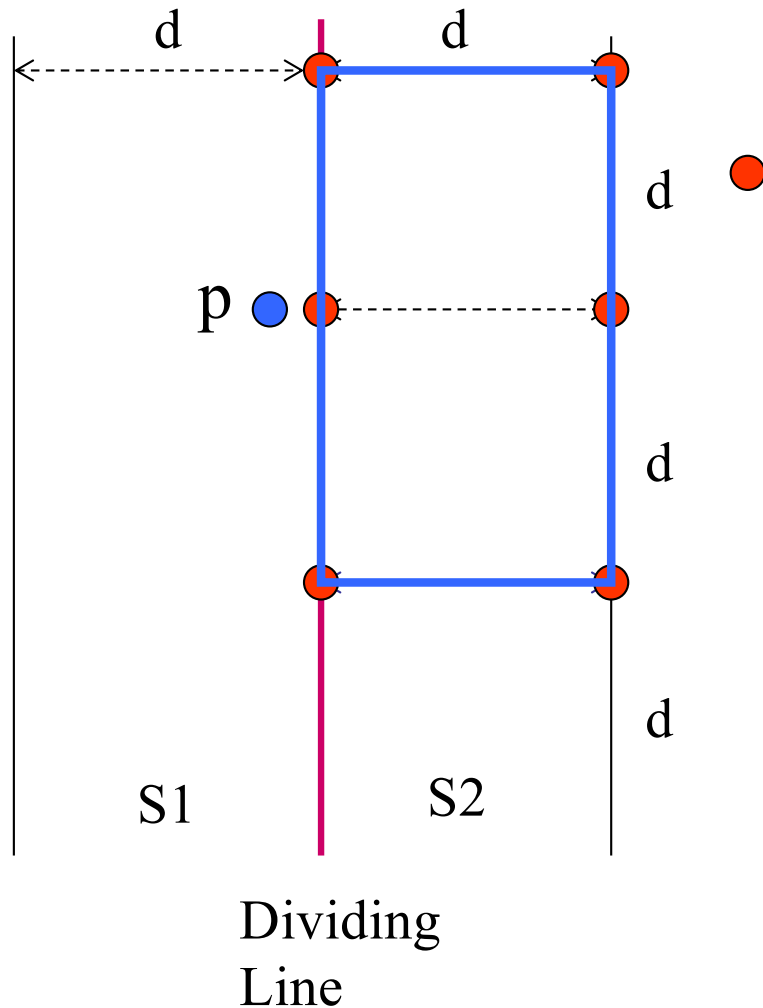
For each point p on the left of the dividing line, we only need to consider the points in the blue rectangle.

Since no two points can be closer than $d = \text{minimum}(d_{Lmin}, d_{Rmin})$, there can only be at most 6 points – 6 red circles in the blue rectangles.

So, we need at most $6 \cdot n/2$ distance comparisons

2D Closest Pair

$$d = \text{minimum}(d_{Lmin}, d_{Rmin})$$



How do we know which 6 points to check?

- Project p and all the points of S_2 within P_2 onto *the red line*.
- Only the points within d of p in the y projection need to be considered (max of 6 points).
- After sorting the points on y coordinate we can find the points by scanning the sorted lists. Points are sorted by y coordinates.

2D Closest Pair

- Time Complexity
 - $T(n) = 2T(n/2) + \theta(n) = \theta(n \lg n)$
 - Solve this recurrence equation yourself by applying the iterative method

Example 4: Strassen's matrix multiplication algorithm

□ Example: 2 by 2 matrix multiplication

$$m1 = (a11 + a22)(b11+b22)$$

$$m2 = (a21 + a22) b11$$

$$m3 = a11 (b12 - b22)$$

$$m4 = a22 (b21 - b11)$$

$$m5 = (a11 + a12) b22$$

$$m6 = (a21 - a11) (b11+b12)$$

$$m7 = (a12 - a22)(b21 + b22)$$

$$C = \begin{bmatrix} c11 & c12 \\ c21 & c22 \end{bmatrix} = \begin{bmatrix} a11 & a12 \\ a21 & a22 \end{bmatrix} \times \begin{bmatrix} b11 & b12 \\ b21 & b22 \end{bmatrix}$$

$$C = \begin{bmatrix} m1 + m4 - m5 + m7 & m3 + m5 \\ m2 + m4 & m1 + m3 - m2 + m6 \end{bmatrix}$$

Example 4 (cont'd)

□ Strassen's matrix multiplication algorithm

- partition $n \times n$ matrix into sub-matrices $n/2 \times n/2$
(assume n is power of 2)
- apply the seven basic calculations:

$$M1 = (A11 + A22)(B11+B22)$$

$$M2 = (A21 + A22) B11$$

$$M3 = A11 (B12 - B22)$$

$$M4 = A22 (B21 - B11)$$

$$M5 = (A11 + A12) B22$$

$$M6 = (A21 - A11) (B11+B12)$$

$$M7 = (A12 - A22)(B21 + B22)$$

$$C = \begin{bmatrix} C11 & C12 \\ C21 & C22 \end{bmatrix} = \begin{bmatrix} A11 & A12 \\ A21 & A22 \end{bmatrix} \times \begin{bmatrix} B11 & B12 \\ B21 & B22 \end{bmatrix}$$

$$C = \begin{bmatrix} M1 + M4 - M5 + M7 & M3 + M5 \\ M2 + M4 & M1 + M3 - M2 + M6 \end{bmatrix}$$

Example 4 (cont'd)

□ Strassen's matrix multiplication example

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \\ 3 & 3 & 4 & 4 \\ 3 & 3 & 4 & 4 \end{bmatrix} \times \begin{bmatrix} 5 & 5 & 6 & 6 \\ 5 & 5 & 6 & 6 \\ 7 & 7 & 8 & 8 \\ 7 & 7 & 8 & 8 \end{bmatrix}$$

$$M_1 = \left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix} \right) \times \left(\begin{bmatrix} 5 & 5 \\ 5 & 5 \end{bmatrix} + \begin{bmatrix} 8 & 8 \\ 8 & 8 \end{bmatrix} \right)$$

$$C = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{bmatrix}$$

Example 4 (cont'd)

□ Strassen's matrix multiplication:

input: n, A, B ;
output: C

```
strassen_matrix_multiply(int n, A, B, C)
{
    divide A into  $A_{11}, A_{12}, A_{21}, A_{22}$ ;
    divide B into  $B_{11}, B_{12}, B_{21}, B_{22}$ ;

    strassen_matrix_multiply( $n/2, A_{11}+A_{22}, B_{11}+B_{22}, M_1$ );
    strassen_matrix_multiply( $n/2, A_{21}+A_{22}, B_{11}, M_2$ );
    .....
    strassen_matrix_multiply( $n/2, A_{12}-A_{22}, B_{21}+B_{22}, M_7$ );

    Compose  $C_{11}, \dots, C_{22}$  by  $M_1, \dots, M_7$ ;
}
```

What is missing in the algorithm?

Example 4 (cont'd)

□ Strassen's matrix multiplication algorithm

- Every case Time complexity

$$\text{Multiplications: } \begin{cases} T(n) = 7 T(n/2); \\ T(1) = 1 \end{cases}$$

$$T(n) = n^{\lg 7} \in \Theta(n^{2.81})$$

(while brute-force approach: $T(n) = \Theta(n^3)$)

$$\text{Additions/Subtractions: } \begin{cases} T(n) = 7 T(n/2) + 18 (n/2)^2 \\ T(1) = 0 \end{cases}$$

$$T(n) = O(n^{2.81}) \text{ by Master theorem } (a=7, b=2, k=2)$$

Example 5: Large Integer Multiplication

□ Large digits are divided into a number of small digits

- $u \times v$

$$\rightarrow u = x \times 10^m + y$$

$$\rightarrow v = w \times 10^m + z$$

$$u \times v = xw \times 10^{2m} + (xz + wy) \times 10^m + yz$$

(split integer (u, v) into two equal size integers $(x, y), (w, z)$)

e.g., $123456 = 123 \times 10^3 + 456$. 123 and 456 all have 3 digits.

Example 5: Large Integer Multiplication

Algorithm 2.9

Large Integer Multiplication

Problem: Multiply two large integers, u and v .

Inputs: large integers u and v .

Outputs: $prod$, the product of u and v .

```
large_integer prod (large_integer u, large_integer v)
{
    large_integer x, y, w, z;
    int n, m;

    n = maximum(number of digits in u, number of digits in v)
    if (u == 0 || v == 0)
        return 0;
    else if (n <= threshold)
        return u × v obtained in the usual way;
    else {
        m = ⌊n/2⌋;
        x = u divide 10m; y = u rem 10m;
        w = v divide 10m; z = v rem 10m;
        return prod(x, w) × 102m + (prod(x, z) + prod(w, y)) × 10m + prod(y, z);
    }
}
```

Example 5: Large Integer Multiplication

□ Worst-case time complexity

$$W(n) = 4 W(n/2) + cn$$

$$W(n) \in \Theta(n^2)$$

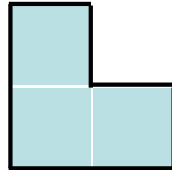
- The linear-time operations of addition, subtraction, divide 10^m , $\text{rem } 10^m$, and $\times 10^m$ all have linear-time complexities in terms of n .

Example 5 (cont'd)

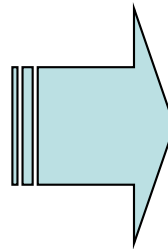
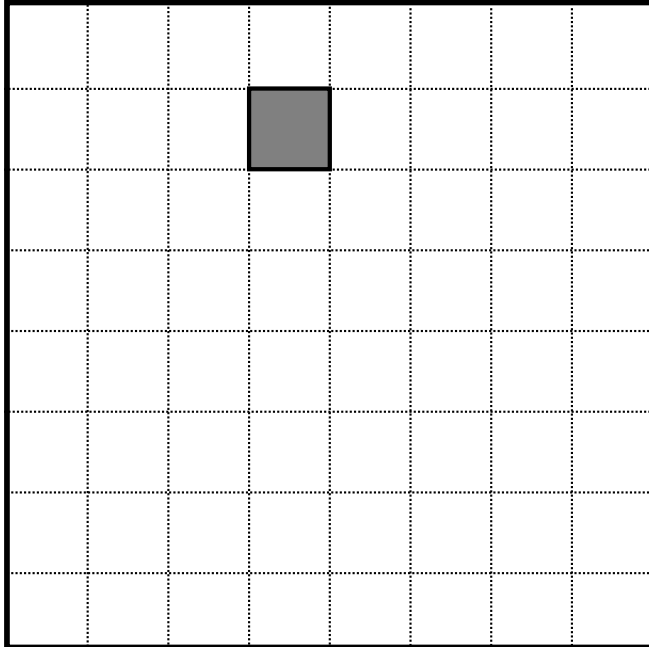
- In the previous solution, we need 4 multiplications xw , $xz + yw$, and yz ; Observe that:
 - $r = (x+y)(w+z) = xw + (xz+yw) + yz$
 - $xz + yw = r - xw - yz$
 - So, just compute $(x+y)(w+z)$, xw , and $yz \rightarrow 3$ multiplications
 - $T(n) = 3T(n/2) + cn$
 - $\Theta(n^{1.58})$ by Master theorem ($a=3, b=2, k=1$)

Example 6: Tromino Tiling

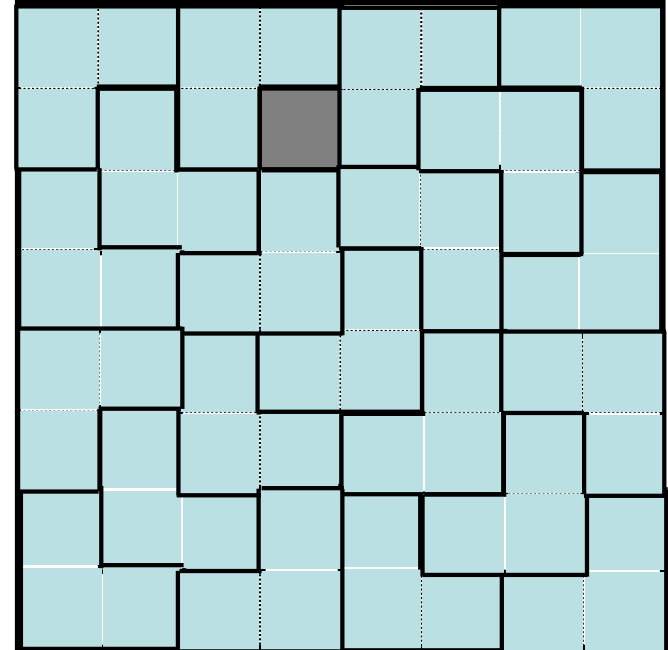
A tromino tile:



And a $n \times n$ ($2^k \times 2^k$) board with a hole:



A tiling of the board with trominos:

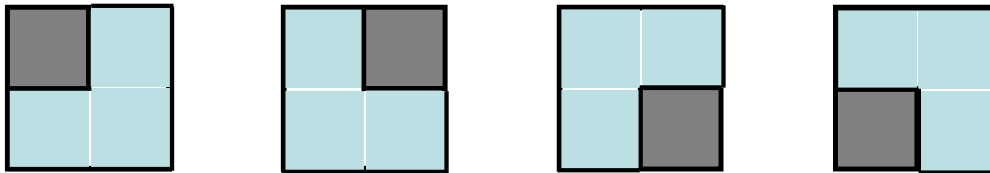


Solvable?

- Total number of squares:
 - $n * n - 1 = 2^k * 2^k - 1 = 2^{2k} - 1 = 4^k - 1$
- Size of one tromino: 3
- So, $4^k - 1$ should be divisible by 3
- Proof by induction
 - Basis: If $n = 2$, $4^1 - 1$ is divisible by 3
 - Induction hypothesis: Assume $4^{k-1} - 1$ is divisible by 3
 - Induction step: $4^k - 1 = 4(4^{k-1} - 1) + 3$

Tiling: Trivial Case ($n = 2$)

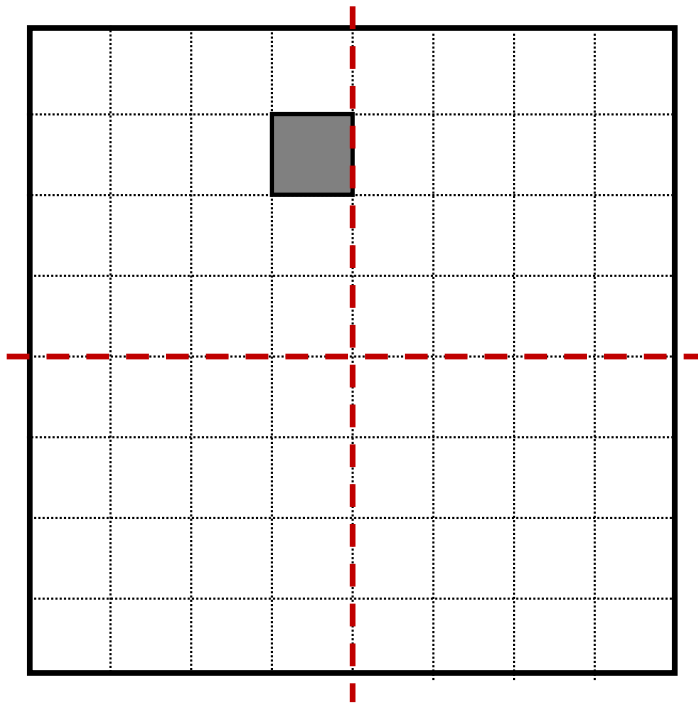
- Trivial case ($n = 2$): tiling a 2×2 board with a hole:



- Idea – try somehow to reduce the size of the original problem, so that we eventually get to the 2×2 boards which we know how to solve...

Tiling: Dividing the Problem

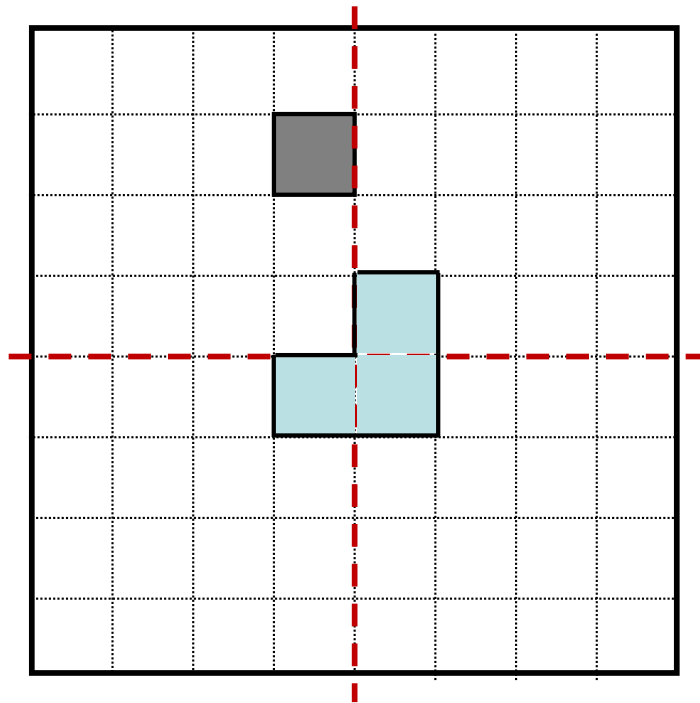
- To get smaller square boards, let's divide the original board into four boards



- Great! We have one problem of the size $(n/2) \times (n/2)$
- But: The other three problems are not similar to the original problems – they do not have holes!

Tiling: Dividing the Problem

- Idea: insert one tromino at the center to get three *imaginary* holes in each of the three smaller boards



- Now we have four boards with holes of the size $(n/2) \times (n/2)$
- Keep doing this division, until we get the 2×2 boards with holes – we know how to tile those

Tiling: Algorithm

INPUT: n – the board size ($2^k \times 2^k$ board where $n = 2^k$), L – location of the hole.

OUTPUT: tiling of the board

Tile(k, L)

if $k = 1$ **then**

Trivial case

 Tile with one tromino

return

 Divide the board into four equal-sized boards

 Place one tromino at the center to cut out 3 additional holes (orientation based on where existing hole, L , is)

 Let L_1, L_2, L_3, L_4 denote the positions of the 4 holes

Tile($k-1, L_1$)

Tile($k-1, L_2$)

Tile($k-1, L_3$)

Tile($k-1, L_4$)

Tiling: Divide-and-Conquer

- Tiling is a divide-and-conquer algorithm:
 - Just do it trivially if the board is 2×2 , else:
 - **Divide** the board into four smaller boards (introduce holes at the corners of the three smaller boards to make them look like original problems)
 - **Conquer** using the same algorithm recursively
 - **Combine** by placing a single tromino in the center to cover the three introduced holes

Time Complexity of Tromino Tiling

- $T(n) = 4T(n/2) + c$
- $T(2) = 1$
- $O(n^2)$ by master theorem (or solve it iteratively)

Combine the D&C algorithm with other simple algorithm

❑ Switching point

- Recursive method may have no advantage for small n compared to an alternative, non-recursive, algorithm
- Recursive algorithm requires a fair amount of overhead
- Stop the dividing process at a certain switching point (or threshold) for recursive algorithm, and then use the alternative algorithm

❑ Example

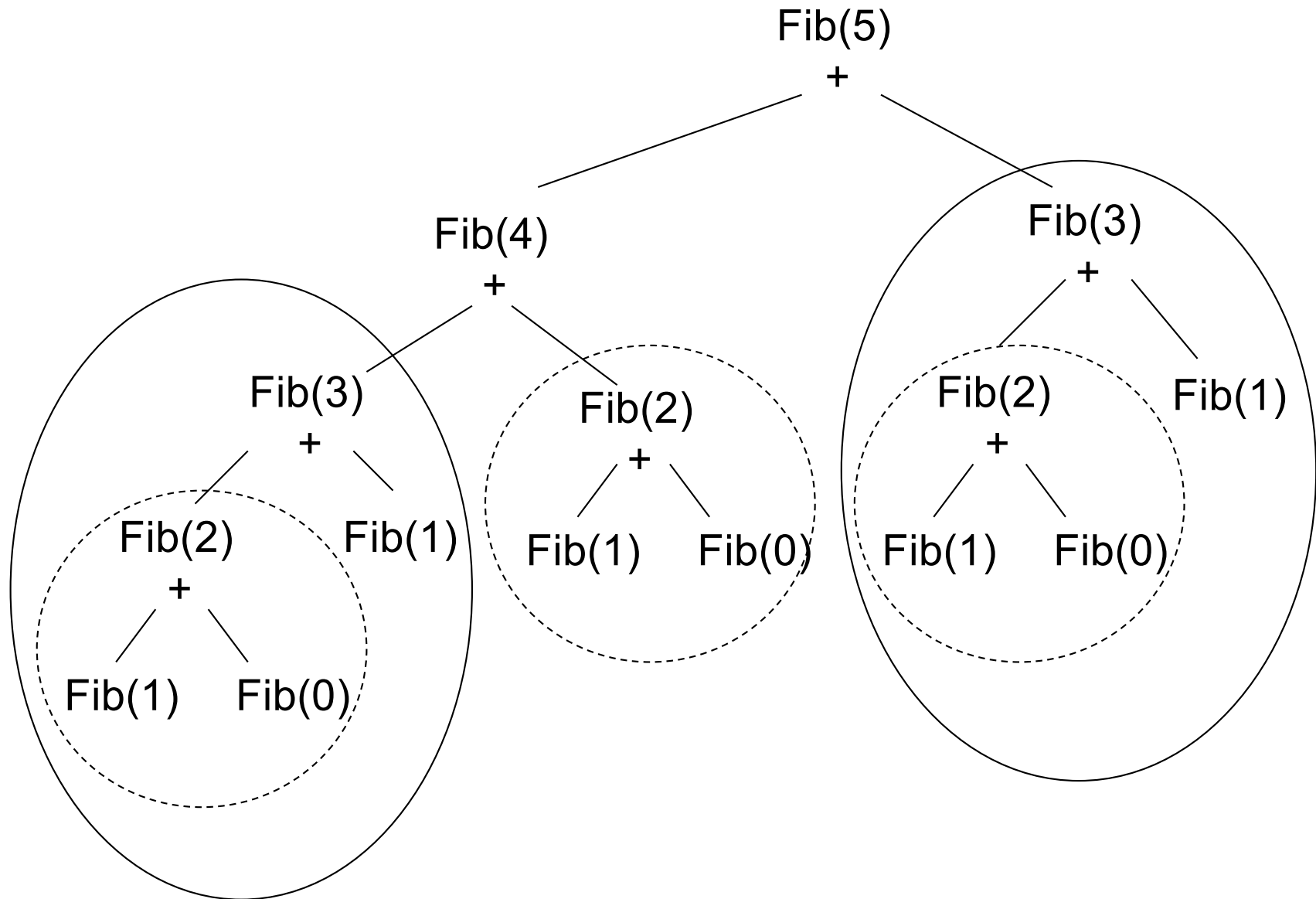
- Use the standard matrix multiplication to multiply two 2×2 matrices

When not to use D&C algorithm

□ Two cases:

- An instance of size n is divided into two or more instances each almost of size n
- An instance of size n is divided into almost n instances of size n/c (c is constant)

Fibonacci number: Divide & Conquer based on recursion is a poor choice!



Fibonacci number: Divide & Conquer based on recursion is a poor choice!

► Theorem 1.1

If $T(n)$ is the number of terms in the recursion tree corresponding to Algorithm 1.6, then, for $n \geq 2$,

$$T(n) > 2^{n/2}.$$

Proof: The proof is by induction on n .

Induction base: We need two base cases because the induction step assumes the results of two previous cases. For $n = 2$ and $n = 3$, the recursion in [Figure 1.2](#) shows that

$$T(2) = 3 > 2 = 2^{2/2}$$

$$T(3) = 5 > 2.8323 \approx 2^{3/2}$$

Induction hypothesis: One way to make the induction hypothesis is to assume that the statement is true for all $m < n$. Then, in the induction step, show that this implies that the statement must be true for n . This technique is used in this proof. Suppose for all m such that $2 \leq m < n$

$$T(m) > 2^{m/2}.$$

Induction step: We must show that $T(n) > 2^{n/2}$. The value of $T(n)$ is the sum of $T(n-1)$ and $T(n-2)$ plus the one node at the root. Therefore,

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + 1 \\ &> 2^{(n-1)/2} + 2^{(n-2)/2} + 1 && \text{by induction hypothesis} \\ &> 2^{(n-2)/2} + 2^{(n-2)/2} = 2 \times 2^{(\frac{n}{2})-1} = 2^{n/2}. \end{aligned}$$